

Resolução do Quebra-Cabeça Tango: Força Bruta e Backtracking como Estudo de Poda em CSPs

Enzo Barcelos¹, Arthur Sita Gomes¹, Rafael Ferraz Barra¹,
Luiz Victor Aldenucci¹, Allan Mateus Arruda de Souza¹, Hugo Macedo de Carvalho¹

¹Curso de Engenharia de Software – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

enzo.barcelos@sga.pucminas.br, arthur.gomes@sga.pucminas.br,
rafael.barra.1384417@sga.pucminas.br, 1352396@sga.pucminas.br,
1399944@sga.pucminas.br, hmcarrvalho@sga.pucminas.br

Resumo. *Este trabalho modela e resolve o quebra-cabeça lógico Tango como um Problema de Satisfação de Restrições, comparando duas técnicas de busca no espaço de estados: força bruta pura e backtracking com poda. As duas estratégias compartilham o mesmo esqueleto recursivo e a mesma lógica de validação das cinco regras do jogo; a diferença está em quando a validação ocorre. A força bruta só verifica o tabuleiro quando ele está inteiramente preenchido, enquanto o backtracking testa cada atribuição na hora e descarta ramos inválidos antes de descer a recursão. Instrumentamos os dois algoritmos com contadores de estados explorados, inclusive por nível da recursão, e medimos o desempenho em sete instâncias, de 4×4 até 10×10 . Os resultados deixam a diferença concreta: numa grade 6×6 com vinte células vazias, a força bruta explora mais de 680 mil estados, contra apenas 31 do backtracking; num 10×10 a força bruta enfrentaria 2^{50} estados (mais de 10^{15}), enquanto o backtracking resolve visitando 151. O perfil por profundidade mostra a raiz da diferença: a força bruta praticamente dobra o número de estados a cada nível, enquanto o backtracking mantém pouco mais de um estado por nível ao longo de toda a árvore.*

1. Introdução

Problemas de Satisfação de Restrições (CSPs) aparecem sempre que precisamos encontrar uma configuração que respeite um conjunto de regras simultâneas. Alocação de horários, coloração de mapas e boa parte dos quebra-cabeças de lógica caem nessa categoria. O atrativo desses jogos para o estudo de algoritmos é que eles têm regras simples de enunciar, mas um espaço de soluções que cresce de forma explosiva com o tamanho do tabuleiro.

O Tango é um desses jogos. Numa grade quadrada de lado par, cada célula recebe um entre dois símbolos (aqui chamados Sol e Lua), e o preenchimento precisa obedecer a cinco regras descritas na Seção 2. A maneira mais direta de atacar o problema é a força bruta: gerar todas as combinações possíveis e ficar com aquela que satisfaz as regras. O número dessas combinações é 2^{N^2} para uma grade $N \times N$, o que já passa de 6×10^{10} num tabuleiro 6×6 . Testar tudo deixa de ser uma opção rapidamente.

O backtracking é o refinamento natural. Em vez de montar o tabuleiro inteiro para só então conferir, ele constrói a solução célula a célula e verifica as regras a cada passo.

Quando uma atribuição já viola alguma regra, o ramo inteiro abaixo dela é abandonado sem ser explorado. Essa poda é o que separa um algoritmo que roda em milissegundos de um que não termina em tempo hábil.

O objetivo deste trabalho é implementar as duas estratégias para o Tango, mantendo a lógica de validação separada da mecânica de recursão, e medir empiricamente o impacto da poda. Duas perguntas guiam a análise. A primeira: a partir de que tamanho a força bruta deixa de ser viável? A segunda: onde, exatamente, a poda economiza trabalho na árvore de busca?

O restante do texto está organizado assim. A Seção 2 descreve como o tabuleiro, as restrições e a validação foram representados. As Seções 3 e 4 detalham as duas estratégias e as comparam estruturalmente. A Seção 5 mostra execuções reais no terminal. A Seção 6 discute o espaço de busca e os resultados medidos, incluindo o perfil da árvore por profundidade. A Seção 7 fecha o trabalho.

2. Modelagem do Problema

2.1. As regras do Tango

O tabuleiro é uma grade $N \times N$ com N par. Cada célula contém Sol ou Lua. As cinco regras são:

1. **Preenchimento completo:** toda célula recebe Sol ou Lua.
2. **Limite de adjacência:** não pode haver três símbolos iguais consecutivos, na horizontal ou na vertical.
3. **Equilíbrio:** cada linha e cada coluna tem exatamente $N/2$ Sóis e $N/2$ Luas.
4. **Igualdade (=):** células ligadas pelo sinal = devem conter o mesmo símbolo.
5. **Oposição (×):** células ligadas pelo sinal × devem conter símbolos opostos.

Um tabuleiro bem formulado tem solução única. As dicas iniciais (algumas células já preenchidas) e as restrições de igualdade e oposição são o que garante essa unicidade.

2.2. Estruturas de dados

A modelagem foi feita em Java, em torno de três tipos no pacote `modelo`. O tipo `Simbolo` é um `enum` com os valores `SOL`, `LUA` e `VAZIO`. Tratar Sol e Lua como opostos binários deixa a regra de oposição e a contagem de equilíbrio bem diretas: o método `oposto()` troca um pelo outro. O tipo `Restricao` é outro `enum`, com `NENHUMA`, `IGUAL` e `OPOSTO`.

O coração da modelagem é a classe `Tabuleiro`, que reúne quatro estruturas:

- `Simbolo[][] grade` guarda o símbolo atual de cada célula, podendo ser `VAZIO` durante a busca;
- `boolean[][] fixa` marca quais células vieram como dica no arquivo de entrada. A busca nunca mexe numa célula fixa;
- `Restricao[][] restricaoH` guarda a restrição entre (r, c) e o vizinho à direita $(r, c + 1)$;
- `Restricao[][] restricaoV` guarda a restrição entre (r, c) e o vizinho de baixo $(r + 1, c)$.

As duas matrizes de restrição têm dimensões diferentes da grade, e isso não é por acaso. Como uma restrição horizontal liga uma célula à sua vizinha da direita, não existe restrição partindo da última coluna; por isso `restricaoH` tem dimensão $N \times (N - 1)$. Pela mesma razão, `restricaoV` é $(N - 1) \times N$. Assim, a restrição entre (r, c) e $(r, c + 1)$ mora exatamente em `restricaoH[r][c]`, e a que liga (r, c) a $(r + 1, c)$ mora em `restricaoV[r][c]`. Esse mapeamento direto evita estruturas auxiliares e torna a consulta de vizinhança uma operação de índice.

Separar grade de fixa foi uma decisão deliberada. Deixa explícito o que é dado do problema e o que a busca pode preencher, e dispensa qualquer estrutura extra para lembrar das dicas durante o backtracking.

2.3. Formato de entrada

A especificação dá liberdade para o formato. Optamos por um arquivo de texto simples. A primeira linha traz N . As N linhas seguintes descrevem a grade, com S para Sol, L para Lua e . para célula vazia. Depois vêm as restrições, uma por linha, no formato `H r c <signal>` (entre (r, c) e a célula à direita) ou `V r c <signal>` (entre (r, c) e a de baixo), com o sinal = ou x. A classe `LeitorTabuleiro` faz a leitura e marca como fixa toda célula que vem preenchida.

2.4. A validação como módulo

A especificação pede que a lógica de validação fique separada da mecânica de recursão. Concentramos as cinco regras numa única classe, o `Validador`, que não sabe nada sobre busca: ela só responde se um estado é válido. As duas estratégias dependem dela, o que garante que ambas julgam o tabuleiro pelos mesmos critérios. São três pontos de entrada:

- `tabuleiroCompletoValido(t)` confere as cinco regras de uma vez, e só faz sentido com o tabuleiro cheio. É o que a força bruta usa.
- `movimentoValido(t, r, c)` verifica apenas o que a célula recém-atribuída (r, c) pode ter quebrado. É o critério de poda do backtracking.
- `dicasValidas(t)` aplica `movimentoValido` a todas as células já preenchidas. O backtracking a chama uma vez, antes de buscar, para garantir que as dicas iniciais não se contradizem — caso contrário, uma violação restrita às células fixas escaparia, já que a busca só percorre as livres.

O custo de cada checagem de `movimentoValido` é baixo. A adjacência olha as janelas de três células que contêm (r, c) , em duas direções, o que é $O(1)$. As restrições de igualdade e oposição comparam (r, c) com até quatro vizinhos, também $O(1)$. O equilíbrio parcial conta o símbolo na linha e na coluna de (r, c) , em $O(N)$. Assim, validar uma jogada custa $O(N)$. A validação completa, por sua vez, varre o tabuleiro inteiro e custa $O(N^2)$.

3. Estratégia de Resolução: Força Bruta

A força bruta corresponde à definição literal de busca exaustiva: gerar o espaço completo de estados e, ao final, verificar quais satisfazem as regras. Na implementação, listamos as células livres uma vez e preenchemos cada uma, em ordem, com Sol e depois com Lua. Nenhuma regra é consultada durante a montagem. A validação acontece só na folha da recursão, quando todas as células já receberam um símbolo.

```

private boolean preencher(Tabuleiro t, int idx) {
    // Caso base: tabuleiro completo. So agora a forca bruta
    // verifica as regras.
    if (idx == livres.length) {
        validacoes++;
        return validador.tabuleiroCompletoValido(t);
    }
    int r = livres[idx][0], c = livres[idx][1];
    for (Simbolo s : new Simbolo[]{Simbolo.SOL, Simbolo.LUA}) {
        t.grade[r][c] = s;
        estados++;
        if (preencher(t, idx + 1)) return true;
    }
    t.grade[r][c] = Simbolo.VAZIO;
    return false;
}

```

A consequência é direta. Com K células vazias, a árvore de recursão tem 2^K folhas, e a validação completa custa $O(N^2)$ em cada uma que é alcançada. O algoritmo só não chega a gerar literalmente todas as 2^K combinações porque para assim que encontra a primeira solução válida. Que isso aconteça cedo ou tarde depende da ordem de enumeração e de onde a solução cai nela, não do número de células vazias. No pior caso, porém, é a árvore inteira que precisa ser percorrida.

4. Estratégia de Resolução: Backtracking

O backtracking usa a mesma espinha dorsal: percorre as células livres em ordem e tenta Sol e depois Lua em cada uma. A diferença está em um ponto só. Logo depois de atribuir um símbolo, ele chama o validador para checar se aquela atribuição já viola alguma regra. Se viola, o ramo é descartado na hora (a poda), e o algoritmo nem tenta descer a recursão a partir dali.

```

public boolean resolver(Tabuleiro t) {
    resetContadores();
    // dicas fixas tambem precisam respeitar as regras (a busca
    // nao as visita)
    if (!validador.dicasValidas(t)) return false;
    livres = t.celulasLivres();
    return preencher(t, 0);
}

private boolean preencher(Tabuleiro t, int idx) {
    if (idx == livres.length) return true; // base: tudo
    // preenchido e valido
    int r = livres[idx][0], c = livres[idx][1];
    for (Simbolo s : new Simbolo[]{Simbolo.SOL, Simbolo.LUA}) {
        t.grade[r][c] = s;
        estados++; validacoes++;
        if (validador.movimentoValido(t, r, c)) {

```

```

        if (preencher(t, idx + 1)) return true;
    } else {
        podas++; // ramo cortado sem recursao
    }
}
t.grade[r][c] = Simbolo.VAZIO; // backtrack
return false;
}

```

A condição de parada (caso base) é alcançada quando todas as células livres foram preenchidas. Como cada passo do caminho já passou pela validação, um caminho que chega ao fim é necessariamente uma solução — desde que as dicas iniciais também respeitem as regras. E aqui há um cuidado: a busca só visita células livres, então `movimentoValido` nunca é chamado sobre uma célula fixa, e uma violação que estivesse inteiramente entre as dicas passaria despercebida. Por isso `resolver` chama `dicasValidas` uma única vez antes de iniciar a busca, conferindo que as dicas são consistentes; com isso, a validação incremental cobre tudo o que a busca toca e a checagem inicial cobre o resto. Não é preciso revalidar o tabuleiro inteiro ao chegar à folha, e há um motivo sutil para isso. O equilíbrio é checado como um limitante: `movimentoValido` rejeita qualquer jogada que faça um símbolo ultrapassar $N/2$ numa linha ou coluna. Ora, se em nenhum momento um símbolo passou de $N/2$ e a linha de N células acabou preenchida, então sobra exatamente $N/2$ de cada um. O equilíbrio exato vem de graça no fim, desde que o limitante tenha sido respeitado o tempo todo.

4.1. Comparação estrutural com a Força Bruta

Vale olhar as duas funções lado a lado, porque elas são quase idênticas. Mesmo laço sobre Sol e Lua, mesma recursão em `idx + 1`, mesmo desfazer no fim. A diferença que importa cabe em uma decisão: *onde* entra a validação.

```

// Força Bruta: valida so quando idx chega ao fim (folha)
if (idx == livres.length) return validador.
    tabuleiroCompletoValido(t);

// Backtracking: valida a cada atribuicao e poda na hora
if (validador.movimentoValido(t, r, c)) { ... } else { podas++;
}

```

Esse deslocamento de uma linha muda o comportamento da árvore de busca por inteiro. Na força bruta, a recursão sempre desce até a folha, então cada nível tem o dobro de nós do anterior, e o número de estados cresce como $2^{\text{nível}}$. No backtracking, a validação na subida corta um ramo cedo, e cortar um ramo no nível i elimina toda a subárvore de tamanho até 2^{K-i} que pendia dele. Quanto mais cedo a poda acontece, maior o galho que cai. A Seção 6.3 mostra esse efeito medido, nível a nível.

5. Exemplos de Execução

O programa opera no terminal. Ao escolher uma instância, ele imprime o tabuleiro inicial, com as lacunas e as restrições desenhadas entre as células, roda as duas estratégias e

imprime o resultado de cada uma. A Figura 1 mostra a resolução de uma grade 6×6 com dez células vazias.

```
>>> Tabuleiro INICIAL (6x6):      ----- Backtracking -----
S   S   L   L   . x .           S   S   L   L   S x L
=                                     =
.   S x L   S   L   L           S   S x L   S   L   L
L   . x .   S   .   S           L   L x S   S   L   S
S   L   S   L   S   L           S   L   S   L   S   L
                                x
L   .   L   S   L   S           L   S   L   S   L   S
L = .   S   .   .   S           L = L   S   L   S   S
                                Estados=15 Validacoes=15 Podas=5
```

Figura 1. Resolução de uma instância 6×6 . À esquerda o tabuleiro inicial com dicas e restrições; à direita a solução do backtracking, que visitou 15 estados.

Para a mesma instância, a força bruta chega exatamente à mesma solução — afinal as duas estratégias enumeram as células na mesma ordem e param na primeira solução válida, então convergem para o mesmo tabuleiro —, mas precisa de 689 estados em vez de 15. A Figura 2 mostra um caso 8×8 , com 35 células vazias. Aqui a força bruta é pulada de propósito: seriam mais de 3×10^{10} estados. O backtracking resolve visitando 254.

```
>>> Tabuleiro INICIAL (8x8):      ----- Backtracking -----
S   .   S   L   .   S   .   .           S   L   S   L   L   S   S   L
S   .   .   .   .   .   .   .           S   S   L   S   L   S   L   L
.   .   .   .   .   L   L   .           L   S   S   L   S   L   L   S
L   L   S   L   S   .   S   S           L   L   S   L   S   L   S   S
                                x
S x . = .   S x .   S   .   L           S x L = L   S x L   S   S   L
x                                     x
L   .   .   L   .   .   .   S           L   S   L   L   S   S   L   S
                                x
.   .   S   S   L   .   .   L           S   L   S   S   L   L   S   L
L   .   L   .   .   L   L   .           L   S   L   S   S   L   L   S
                                (Força Bruta pulada: ~2^35 estados)
```

Figura 2. Resolução de uma instância 8×8 com 35 células vazias. A força bruta é inviável; o backtracking resolve em 254 estados.

Para fechar a faixa de tamanhos, o programa também resolve um 10×10 com 50 células vazias. A força bruta enfrentaria 2^{50} estados, algo em torno de 10^{15} . O backtracking devolve a solução visitando 151 estados, em frações de milissegundo.

6. Análise de Complexidade

6.1. O tamanho do espaço de busca

Com dois símbolos por célula, uma grade $N \times N$ admite 2^{N^2} preenchimentos. São cerca de $6,9 \times 10^{10}$ para 6×6 e $1,8 \times 10^{19}$ para 8×8 . Quando parte das células já vem preenchida como dica, o que importa é o número K de células livres, e o espaço cai para 2^K .

As regras do jogo recortam esse espaço bem antes de a busca terminar. A de maior efeito é o equilíbrio. Uma linha válida tem exatamente $N/2$ de cada símbolo,

então o número de linhas possíveis não é 2^N , mas $\binom{N}{N/2}$. Considerando apenas essa regra, o número de tabuleiros candidatos cai de 2^{N^2} para $\left(\binom{N}{N/2}\right)^N$. Em 6×6 isso significa $20^6 \approx 6,4 \times 10^7$, contra $6,9 \times 10^{10}$ do espaço bruto, uma redução de três ordens de grandeza só com uma das cinco regras. Em 8×8 , $\binom{8}{4}^8 = 70^8 \approx 5,8 \times 10^{14}$, contra $1,8 \times 10^{19}$.

A adjacência e as restrições de igualdade e oposição cortam ainda mais. O ponto que interessa para a comparação é *quando* esse corte é aplicado. A força bruta enxerga as regras apenas no fim, depois de pagar o custo de montar o tabuleiro inteiro. O backtracking as aplica na construção, e por isso nunca chega a montar a maioria dos tabuleiros que a força bruta gera.

6.2. Resultados medidos

A Tabela 1 traz os contadores de estados para as sete instâncias. O número de estados é determinístico: depende só da instância e da ordem de preenchimento, não do relógio nem da máquina. É a métrica mais confiável para comparar as duas estratégias.

Tabela 1. Estados explorados por cada estratégia. Onde a força bruta é inviável, reportamos o limite teórico 2^K .

Instância	N	K	BT estados	BT podas	FB estados
4×4 didático	4	8	12	4	408
6×6 fácil	6	10	15	5	689
6×6 médio	6	20	31	10	683.347
6×6 vazio	6	22	32	9	427.862
8×8 difícil	8	35	254	118	$\approx 2^{35} = 3,4 \times 10^{10}$
8×8 esparso	8	45	298	138	$\approx 2^{45} = 3,5 \times 10^{13}$
10×10 grande	10	50	151	64	$\approx 2^{50} = 1,1 \times 10^{15}$

A Figura 3 põe esses números num gráfico de escala logarítmica, com o número de células vazias K no eixo horizontal. A reta pontilhada marca o espaço de busca 2^K : a força bruta acompanha esse limite enquanto é viável, enquanto o backtracking corre ordens de grandeza abaixo.

Dois detalhes da tabela merecem comentário, porque contrariam a intuição de que “mais células vazias significa mais trabalho”. O 6×6 vazio tem mais células livres que o médio (22 contra 20), mas a força bruta explora menos estados nele (427.862 contra 683.347): é que a primeira solução, na ordem de enumeração, aparece mais cedo nessa instância. Do lado do backtracking, o 10×10 (151 estados) dá menos trabalho que o 8×8 esparso (298), apesar de ser maior. O motivo é que o esforço do backtracking não depende diretamente de K , e sim de quão restrita é a instância: dicas e restrições bem postas forçam as podas a aparecerem cedo. Vale registrar que as três instâncias maiores e mais esparsas (vazio, esparso e grande) não foram construídas para ter solução única — elas servem para medir o custo de busca —, ao contrário das instâncias didáticas menores, geradas com verificação de unicidade. A Figura 4 compara os dois lado a lado, e a Figura 5 mostra quantas vezes a força bruta explora mais estados.

6.3. Perfil da árvore de busca por profundidade

A Tabela 1 mostra o total de estados, mas não *onde* o trabalho se concentra. Para enxergar isso, instrumentamos os dois algoritmos para contar quantos estados são explorados em

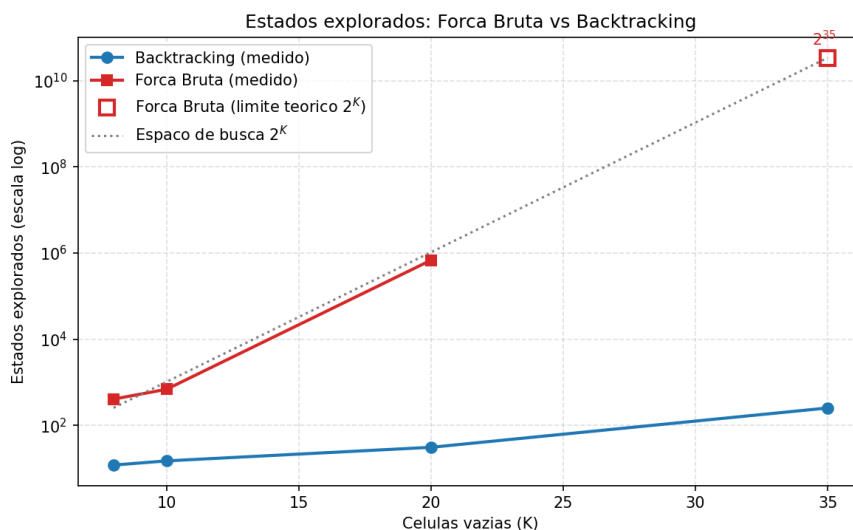


Figura 3. Estados explorados em função do número de células vazias K (escala logarítmica). A força bruta acompanha o espaço de busca 2^K (pontos vazados são o limite teórico, onde ela é inviável); o backtracking cresce muito mais devagar.

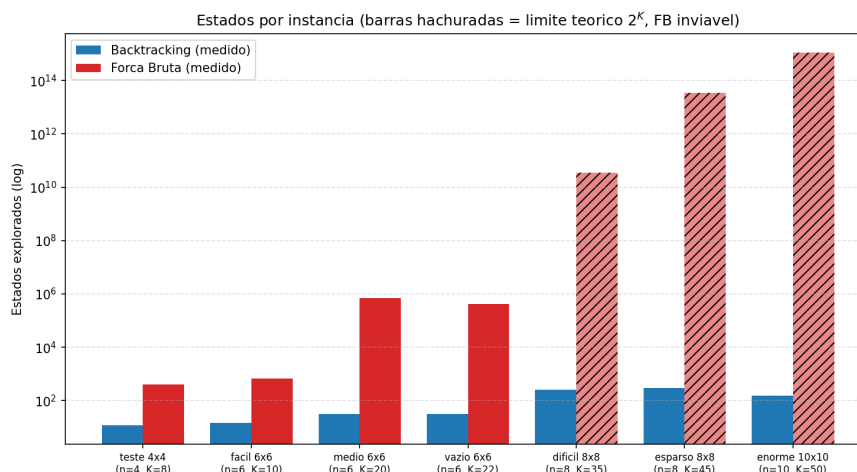


Figura 4. Estados explorados por instância. As barras hachuradas são o limite teórico 2^K , nos casos em que a força bruta é inviável.

cada nível da recursão. A Figura 6 traz o resultado para o 6×6 médio, a maior instância em que as duas estratégias terminam.

A leitura do gráfico é o resumo de todo o trabalho. A curva da força bruta é uma reta na escala logarítmica: o número de estados praticamente dobra a cada nível, chegando a mais de 340 mil no último. É o crescimento $2^{\text{nível}}$ esperado de uma árvore binária percorrida sem poda. A curva do backtracking, em contraste, fica colada à base, tipicamente em um ou dois estados por nível, com um pico isolado de três logo no início. A poda mantém a busca quase como um único caminho descendo até a solução. Aquela diferença de uma linha entre as duas funções, discutida na Seção 4.1, é exatamente o que separa as duas curvas.

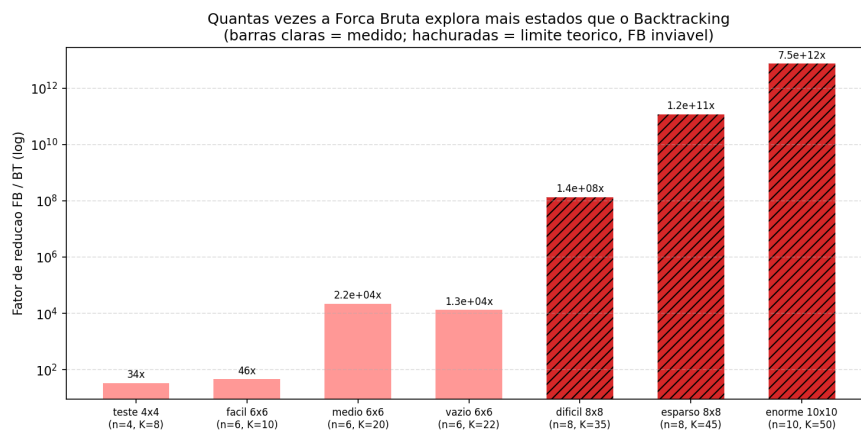


Figura 5. Fator de redução: número de estados da força bruta dividido pelo do backtracking. Cresce de $34\times$ no 4×4 a mais de $10^{12}\times$ no 10×10 .

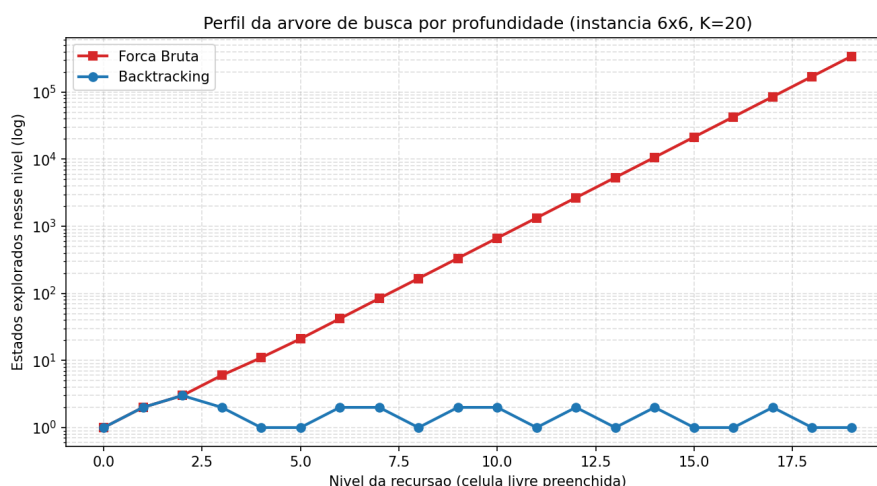


Figura 6. Estados explorados em cada nível da recursão (6×6 , $K = 20$). A força bruta dobra a cada nível; o backtracking fica tipicamente em um ou dois estados por nível.

6.4. Tempo de execução

A Tabela 2 mostra os tempos, medidos após uma execução de aquecimento e tirando a média de cinco repetições, com o desvio padrão amostral. Nas instâncias menores os tempos são tão baixos que ficam dominados por ruído de medição, e por isso a leitura honesta vem dos estados, não do relógio. O 6×6 médio é o que dá o sinal mais claro: a força bruta leva cerca de 9 ms, contra centésimos de milissegundo do backtracking.

A Figura 7 traz os mesmos tempos em escala logarítmica. Confirma a ressalva: nas instâncias pequenas as barras de erro engolem a diferença, e só a partir do 6×6 médio o tempo passa a contar a mesma história que os estados.

6.5. Complexidade assintótica

No pior caso, as duas estratégias são exponenciais: nada impede uma instância patológica de forçar o backtracking a explorar quase toda a árvore. A força bruta faz $O(2^K)$

Tabela 2. Tempo de execução em milissegundos (média $\pm$ desvio padrão amostral sobre cinco repetições, após aquecimento).

Instância	K	Backtracking (ms)	Força Bruta (ms)
4×4 didático	8	0,027 $\pm$ 0,005	0,073 $\pm$ 0,038
6×6 fácil	10	0,026 $\pm$ 0,005	0,041 $\pm$ 0,003
6×6 médio	20	0,016 $\pm$ 0,001	9,040 $\pm$ 1,569
6×6 vazio	22	0,017 $\pm$ 0,001	5,466 $\pm$ 0,124
8×8 difícil	35	0,058 $\pm$ 0,020	(inviável)
8×8 esperso	45	0,053 $\pm$ 0,003	(inviável)
10×10 grande	50	0,044 $\pm$ 0,002	(inviável)

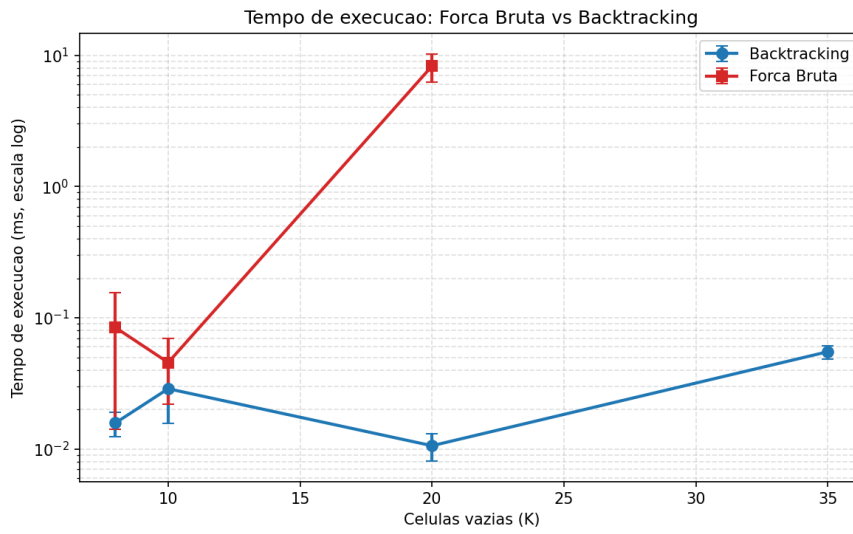


Figura 7. Tempo de execução em função de K (escala logarítmica, barras de erro do desvio padrão amostral). Nas instâncias pequenas os tempos se confundem com o ruído de medição; a separação só fica nítida a partir do 6×6 médio.

construções — $\Theta(2^K)$ quando a solução está no fim da enumeração ou não existe — somadas a $O(N^2)$ por validação na folha. O backtracking tem o mesmo limite superior assintótico, com uma diferença que muda tudo na prática: cada poda no nível i elimina uma subárvore de tamanho até 2^{K-i} . Em tabuleiros bem formulados, as violações aparecem cedo e em quantidade, e o número de estados efetivamente visitados fica muito abaixo do pior caso. É essa distância entre o pior caso teórico e o comportamento típico que os números das tabelas e o perfil por profundidade evidenciam.

7. Conclusão

A primeira pergunta da introdução era a partir de que tamanho a força bruta deixa de servir. Os experimentos respondem de forma concreta. Já num 6×6 com vinte células vazias a força bruta precisa de centenas de milhares de estados, e a partir do 8×8 ela passa de 10^{10} , o que a torna inútil na prática. O backtracking resolve as mesmas instâncias, e ainda um 10×10, visitando de algumas dezenas a poucas centenas de estados.

A segunda pergunta era onde a poda economiza. O perfil por profundidade mostrou com clareza: a força bruta dobra o trabalho a cada nível, enquanto o backtracking

mantém pouco mais de um estado por nível, porque corta os ramos inválidos antes de descer. A poda não muda a classe de complexidade no pior caso, mas é a diferença entre um algoritmo que termina instantaneamente e um que não termina.

O ganho vem inteiro de uma decisão de projeto: validar durante a construção, não no fim. Manter as cinco regras isoladas num `Validador` deixou as duas estratégias usarem exatamente a mesma lógica, com a força bruta chamando a versão completa e o backtracking a versão incremental. Isso tornou a comparação justa e o código fácil de conferir. Como continuação, duas direções aparecem naturais: escolher a próxima célula pela heurística do mais restrito primeiro (MRV), para antecipar as podas, e propagar restrições deduzindo valores forçados antes de tentar, aproximando o resolvidor da forma como uma pessoa joga Tango.